Final Project

Overview

Your team is challenged with designing and building a financial application based on your chosen project specification. Your task is to build the application.

Technical Goals

You should aim to create a REST API for your application. This API should allow saving and retrieving records that describe the core entities of your system.

Integration Requirement: Every project should have AT LEAST one external API integration.

Bonus Objective: One nice extra is to expose an API to external users as well (instructors could use this API to check some information from the system during the final presentation).

If/When you have made progress on the core requirements, requirements for further enhancements will be provided. This will include open-ended enhancements whereby you can make use of your particular skills and experience.

For the project, you can use the technologies you have learnt about in your training, utilizing **Java, JavaScript, or TypeScript**. If you wish to use a specific framework or some other technology (such as React, Vue.js etc.), please check with your instructor.

The Front end should facilitate your users to (in order of priority):

- Browse core system records
- View relevant metrics or performance (ideally in some graphical manner)
- Add items to the system
- Remove items from the system

In terms of detailed requirements, your instructor will act as a customer, and will tell you what they want. You can arrange meetings with them as required.

Notes

1. User management is something you CAN do, but you don't have to do if you don't have time. A single user can be assumed initially.
2. You should use the database technology you have been using in the training for any persistent storage, but other database solutions are also acceptable.
3. Make good use of git. Use branching and pull requests if you can.
4. Any documentation about how to use your REST API would be useful. Maybe Swagger if you have covered it in your training.
5. These technologies and solutions are strongly suggested: CI/CD, Docker, Unit tests, and End-to-End tests.

Technical Getting Started Checklist

1. Create your project structure.
2. Create a GitHub repository. Everybody should use GitHub as a version control system. Instructors should be invited to join the project as viewers (usernames: **helppo2** and **tuistmessiah**).
3. Set up Trello, Jira, or any chosen project management tool to keep track of your tasks and progress.
4. Add, commit, push your skeleton project to your Git repository.
5. Ensure your team has access to the Git repository.
6. Decide on the absolute MINIMUM fields for a first working system (e.g., the first version of your model object may just be an id, a primary identifier like a ticker, and a volume/value).

If you get stuck getting any of the above completed then contact your instructor for help.

Project Management Getting Started Checklist

1. As a team decide how you will approach the work. E.g. 2 people on the backend, 1 person on UI Design Vs. Everyone on the backend until a basic system is working.
2. Make a task list. Ideally use a tool such as trello to keep track of tasks.
3. Some of your team may work on the DESIGN of a more fully-featured application. While some of your team work on BUILDING some small pieces as demonstration.
4. Choose the tasks required for a MINIMAL implementation first.
5. Your instructor will drop in regularly to see how you're progressing. Make a note of any questions so that you're ready to ask them then.

6. Your team should get together and decide on an initial set of data that you will store. A good team decision on this is a good path to success, however remember to STAY AGILE. The single biggest problem teams face is starting out with a data model that is too complex.

Suggestions for Success

1. START SMALL. Get a system working that stores a very simple object with minimal fields. You can then enhance to store more complex records.
2. Try pair programming, it can be very effective.
3. Take conscious steps to keep a good energy in the team. E.g. give your team a name, systematically plan check-ins with each other.
4. Emphasize quality over quantity.

Project Presentations

At the end of the program you will get the opportunity to present your project to your instructors and also potentially your manager and other interested stakeholders from within the firm.

The presentation and demo time is 15 minutes + 5 minutes for questions and answers. This applies to all groups regardless of the group size.

Presentation Guidelines

Here is a suggested flow. You don't have to follow this exactly, but it gives you a suggested outline:

- Tell a story!
- Your presentation should have a beginning, a middle and an end
- Start by introducing your team
- Then introduce the project
- What have you been learning?
- What were you asked to do?
- How much time have you had to work on it?
- Then explain how you approached the project
- Did you divide roles, e.g. backend or frontend?
- Or did you code together, e.g. pair-programming?
- What technologies and tools did you use?
- Then show what you built
- Start with an overview of your data model – explain your decisions

- Then show a high-level architecture of your application
- This could be a simple diagram in PowerPoint
- Then give us a live demonstration of your application
- Then tell us what challenges you faced
- Did you work well together as a team?
- Were there any technical challenges?
- What mistakes did you make?
- What would you do differently?
- Then tell us what you would do next if you had more time
- And finally – thank you for listening, any questions
- Everyone is expected to speak
- Keep your cameras on throughout the presentation
- NOTE: YOU WILL BE EXPECTED TO ASK OTHER TEAMS QUESTIONS

Appendix A: Notes on Teamwork

It is expected that you work closely as a team during this project.

Your team should be self-organising, but should raise issues with instructors if they are potential blockers to progress.

Your team can use a task management system such as Trello to keep track of tasks and progress. Divide the work

Your team should keep track of all source code with git.

Each project team should have only one repository that contains all work.

Your instructor and team members need to access all repositories, so they should be either:
a) Made public
b) Shared with your instructor and all team members.

Throughout your work, you should ensure good communication and organise regular check-ins with each other.

Project Specifications

1. Equity Trade Booking Engine

- **Base Requirement:** Books stock trades and calculates current unrealized P&L using live stock prices.
- **Implementation Details:** Build a web frontend for trade entry and a backend that validates inputs, stores trade data in a SQL database, and utilizes data synchronization to cache live stock prices to prevent rate-limiting.
- **Presentation Hook:** Demonstrate the system's resilience by simulating an external API outage and showing how the backend smoothly falls back to cached pricing data.
- **Suggested Free APIs:** Alpha Vantage or Finnhub provide excellent free tiers for real-time US equity quotes.

2. Order Management System (OMS)

- **Base Requirement:** Manages buy and sell orders, validates them against the latest market prices, and retrieves real-time stock quotes.
- **Implementation Details:** Implement a state machine in the backend (e.g., Pending, Executed, Rejected) and use authentication (API keys) to ensure only authorized users can approve large orders.
- **Presentation Hook:** Show the real-time order validation flow, highlighting how the system blocks "stale" limit orders when the market price suddenly drops.
- **Suggested Free APIs:** Polygon.io provides a robust basic free tier for market data, while Twelve Data offers real-time quotes suitable for an OMS.

3. Portfolio Manager

- **Base Requirement:** Users manage portfolios of stocks and ETFs, while portfolio values, gains/losses, and allocations are updated using live market prices.
- **Implementation Details:** Create interactive frontend charts (using libraries like Chart.js or D3) to visualize asset allocation, supported by a backend polling engine that fetches daily closing prices.
- **Presentation Hook:** Pitch the product as a "client-ready" dashboard, emphasizing how working with real-world data creates a seamless user experience compared to static toy examples.
- **Suggested Free APIs:** Yahoo Finance (via open-source libraries like `yfinance`) or Twelve Data are highly reliable for historical ETF and stock data.

4. Bond Portfolio Manager

- **Base Requirement:** Tracks bond holdings and maturity dates, using public interest rate or bond yield data to estimate portfolio value.
- **Implementation Details:** Build a backend cron job to sync daily yield curves and apply basic fixed-income math to estimate price shifts against the bonds' par values and coupons.
- **Presentation Hook:** Focus the demo on macroeconomic integration, showing how changes in national interest rates automatically cascade into the portfolio's estimated valuation.
- **Suggested Free APIs:** The FRED API (Federal Reserve Economic Data) offers free treasury yields (e.g., DGS10 for 10-year yield) and requires only a free API key. The ECB (European Central Bank) also offers free public datasets.

5. Portfolio Rebalancing Service

- **Base Requirement:** Compares the current portfolio against target allocations using live asset prices and recommends trades to rebalance the portfolio.
- **Implementation Details:** Implement an algorithmic engine that calculates the delta between current and target weightings, generating an automated list of "suggested buy/sell tickets" required to restore balance.
- **Presentation Hook:** Highlight the automation logic and efficiency gains, presenting the service as a tool that saves wealth managers hours of manual spreadsheet calculations.
- **Suggested Free APIs:** CoinGecko is fantastic for a crypto-focused rebalancer, or Alpha Vantage for traditional equities.

6. Market Watch Dashboard

- **Base Requirement:** Tracks a watchlist of stocks, ETFs, or cryptocurrencies, displaying live prices, daily changes, charts, and configurable alerts.
- **Implementation Details:** Develop a highly responsive single-page application (SPA) and implement a scheduled backend job that triggers email or in-app alerts when an asset crosses a user-defined price threshold.
- **Presentation Hook:** Show off the real-time alerting capability by manually pushing a price update to trigger a live "sudden drop" notification during the demo.
- **Suggested Free APIs:** CoinGecko offers an extensive, free, no-key-required API for cryptocurrency tracking. Finnhub is great for stock watchlists.

7. Corporate Actions Processor

- **Base Requirement:** Maintains stock holdings and processes dividends and stock splits using publicly available corporate action data.
- **Implementation Details:** Design a database schema that can retroactively apply multipliers (for splits) or cash additions (for dividends) to historical trade records without corrupting the original entries.
- **Presentation Hook:** Walk managers through the complexity of a 3-for-1 stock split and demonstrate how the system mathematically adjusts user holdings seamlessly in the background.
- **Suggested Free APIs:** Alpha Vantage and Finnhub provide free endpoints specifically dedicated to historical dividends and stock splits.

8. Instrument Reference Data Service

- **Base Requirement:** Maintains a searchable catalog of stocks, ETFs, and companies, synchronizing instrument metadata from an external API.
- **Implementation Details:** Build a lightweight microservice focused on search indexing, utilizing background polling to periodically update company names, tickers, and sector classifications.
- **Presentation Hook:** Position the project as an internal "golden source of truth" that other developers in the organization can consume via a REST API.
- **Suggested Free APIs:** Finnhub offers a wealth of free company fundamentals and metadata alongside its pricing data.

9. Personal Investment Tracker

- **Base Requirement:** Users record purchases and sales of investments, while the system calculates current portfolio value, profit/loss, and performance using live market data.
- **Implementation Details:** Focus on consumer-grade UI/UX with strict user authentication, ensuring that individual user data is segregated at the database level.
- **Presentation Hook:** Highlight the platform's potential as a B2C application by walking through a new user's onboarding journey and their first trade entry.
- **Suggested Free APIs:** Twelve Data and Polygon.io both offer excellent basic tiers for tracking daily equity and crypto movements.

10. Simple Online Banking Service

- **Base Requirement:** Users manage accounts and transfers through a web application that displays balances in multiple currencies using live exchange rates.

- **Implementation Details:** Implement strict transactional database locks to prevent double-spending during transfers, and build a currency conversion utility utilizing live FX APIs.
- **Presentation Hook:** Demonstrate a cross-border transfer simulation, emphasizing how the system locks in the current exchange rate at the exact moment of execution.
- **Suggested Free APIs:** Frankfurter is a free, open-source currency API that tracks daily exchange rates and requires no API key. ExchangeRate.host is another robust alternative.

11. Payment Reconciliation Engine

- **Base Requirement:** Compares payment records from multiple systems, detects differences, and enriches transactions with current FX rates for cross-currency reconciliation.
- **Implementation Details:** Create a backend script that ingests two separate CSV or JSON ledgers, matches records based on fuzzy logic or IDs, and flags discrepancies in a dedicated review dashboard.
- **Presentation Hook:** Pitch this as an operational risk-reduction tool, showing how easily the system caught a simulated "missing $50,000 payment" during testing.
- **Suggested Free APIs:** Frankfurter or the ECB API are ideal for fetching the historical exchange rates needed to reconcile past cross-currency payments.

12. Suspicious Activity Monitor

- **Base Requirement:** Detects unusual transactions using configurable rules and enriches transactions with customer risk scores, sanctions lists, or country information from external services.
- **Implementation Details:** Develop a rules-engine backend (e.g., flagging transactions over $10k or transfers to high-risk jurisdictions) and integrate a third-party API to evaluate the counterparty's risk profile.
- **Presentation Hook:** Run a live simulation where a standard transaction passes, but a transfer to a sanctioned entity is immediately quarantined by the system.
- **Suggested Free APIs:** OpenSanctions provides a free bulk database download for non-commercial and academic use. The free REST Countries API can be used to enrich geographic data.

13. Trade Lifecycle Tracker

- **Base Requirement:** Tracks trades from booking to settlement while displaying current market values using live prices.

- **Implementation Details:** Build an event-driven architecture that moves a trade through T+1 or T+2 settlement phases, updating the database status via scheduled cron jobs.
- **Presentation Hook:** Show a timeline visualization of a trade's lifecycle, emphasizing the operational transparency the tool provides to middle-office teams.
- **Suggested Free APIs:** Yahoo Finance or Alpha Vantage can supply the end-of-day pricing necessary to value the trade at different stages of settlement.

14. Transaction Audit Trail Service

- **Base Requirement:** Records every business operation, provides searchable history, and enriches audit records with market prices or exchange rates captured at execution time.
- **Implementation Details:** Design an immutable, append-only database structure. Use API integration to timestamp and snapshot external market conditions at the exact second a transaction is logged.
- **Presentation Hook:** Emphasize compliance and security, demonstrating how the system guarantees that historical financial context cannot be tampered with once recorded.
- **Suggested Free APIs:** Frankfurter (for precise FX snapshots) or Finnhub (for equity pricing snapshots).

15. Financial News & Portfolio Impact Service

- **Base Requirement:** Aggregates financial news for companies in a portfolio and estimates which holdings are affected by integrating with both a financial news API and a stock price API.
- **Implementation Details:** Use natural language processing (via local open-source models or AI APIs) to run basic sentiment analysis on fetched news headlines, correlating positive/negative news with daily stock price shifts.
- **Presentation Hook:** Show a side-by-side view of a major breaking news headline and the corresponding real-time dip or spike in a related portfolio asset.
- **Suggested Free APIs:** Finnhub provides up to one year of historical news and updates for North American companies on its free tier.